



SHNOOR™
INTERNATIONAL LLC

AI- Plagiarism Detection System

Project Documentation

Developed By: SHNOOR INTERNATIONAL LLC

About Plagiarism Detection System

AI-Powered Plagiarism Detection System is a professional code-authenticity platform for analyzing pasted source code and uploaded files through plagiarism similarity scoring, AI-generated logic assessment, secure user workflows, and history-based review in one connected experience.

Technology Stack

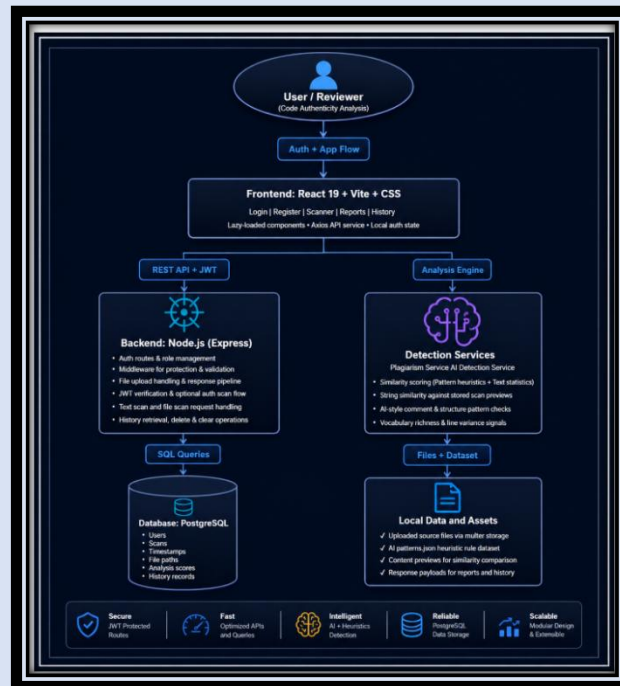
Frontend: Built with React 19 and Vite for a fast, component-based interface. It delivers login, registration, scanner actions, result views, modal reports, and responsive history management through integrated API communication.

Backend: : A Node.js and Express application handles secure API routing, authentication, scanning workflows, file processing, persistence, and report generation logic for the platform.

Detection Engine: The intelligence layer combines string-similarity based comparison with local AI-style heuristics, text statistics, and configurable pattern datasets to produce practical analysis outcomes.

Database: PostgreSQL stores users and scan records, including content previews, scores, timestamps, and user-linked history for traceable review workflows.

System Architecture



Tier 1 : User Interface Layer (Frontend)

Built using React 18 and Vite, this layer delivers a responsive dashboard for secure login, document upload, direct text input, real-time AI content detection, plagiarism scanning, comprehensive report generation, and historical scan viewing through REST API integration.

Tier 2 : Business Logic Layer (Backend)

Implemented with Node.js, Express, and specialized service modules, this layer handles JWT authentication, document ingestion, text parsing, user session management, scanning history operations, isolated sandbox environments, and core originality comparison logic routing.

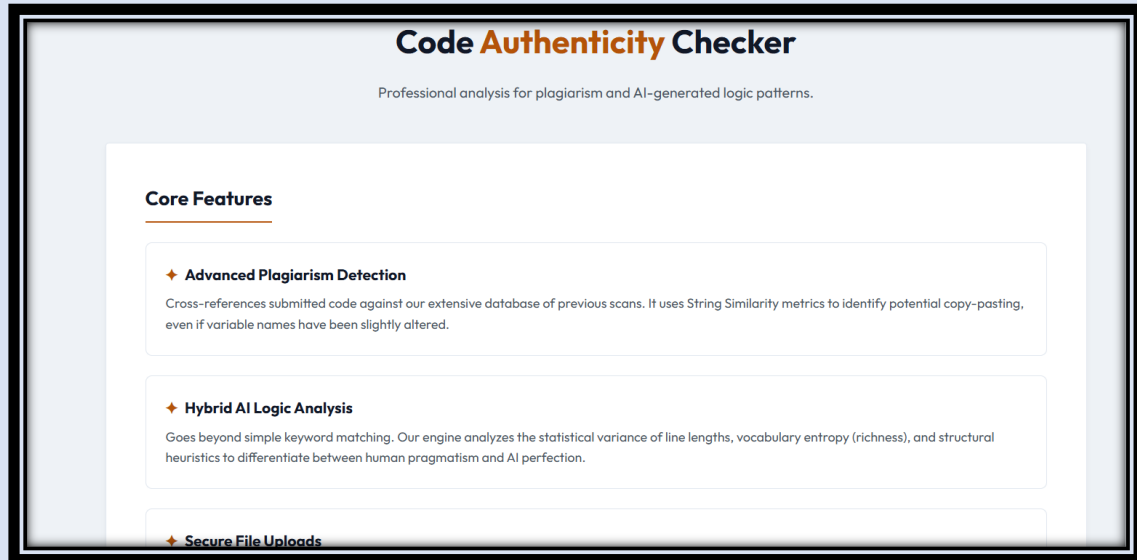
Tier 3 : AI Intelligence Layer

Powered by Node.js and custom JSON pattern matching datasets, this layer provides vocabulary richness analysis, AI structure evaluation, and similarity scoring using string-similarity algorithms. It supplies intelligent originality metrics, text statistics, and authorship confidence labels.

Tier 4 : Database Layer

Implemented with PostgreSQL, this layer stores user profiles, scanned document previews, extracted text, plagiarism scores, AI detection results, and user-specific scan histories. It supports persistent historical records and isolated sandboxed comparison references.

Landing Page



Features Overview

It presents the platform's capabilities via the dedicated Features page, highlighting real-time text scanning, AI generation detection, and detailed plagiarism reporting.

Header Navigation

The persistent header introduces the Shnoor corporate branding and seamlessly guides visitors between the Home dashboard, About, Features, and Help sections.

Access & State Management

The header actively monitors authentication state, dynamically revealing the "Home" tab and "Logout" button only when users have an active JWT session.

Brand Identity

The navigation bar establishes the platform's identity using the official Shnoor logo and professional color scheme, maintaining visual consistency across all views.

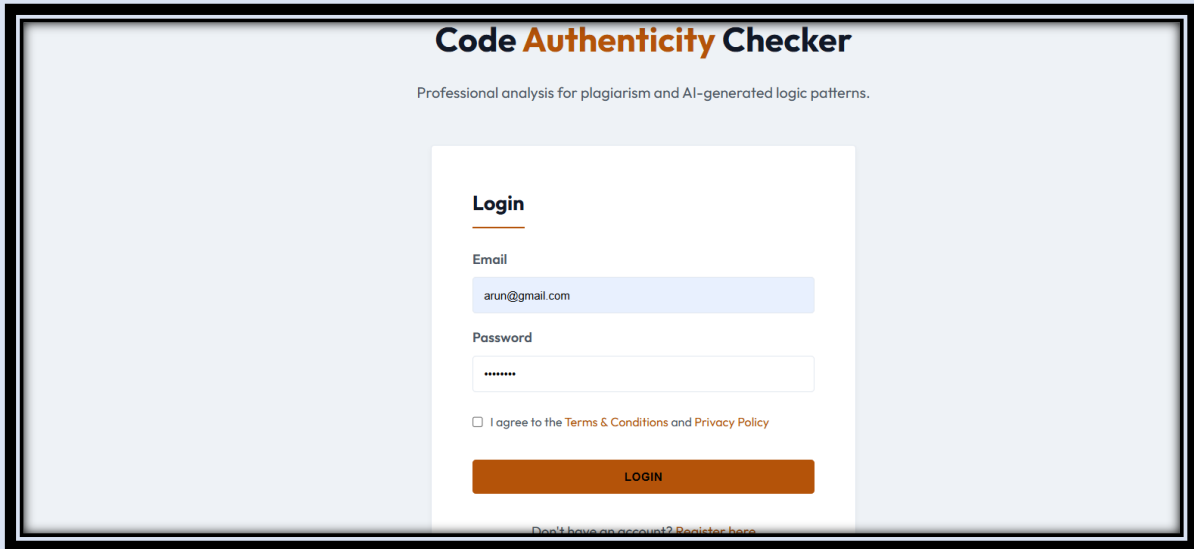
Informational Gateway

The About and Help components function as the opening gateway, educating users on how the detection engine works before they authenticate and initiate scans.

Navigation Intent

The top-level tab design reduces onboarding friction by making the platform's core informational pages and authentication actions immediately visible and accessible at all times.

Login Page



The screenshot shows a web interface titled "Code Authenticity Checker" with the subtitle "Professional analysis for plagiarism and AI-generated logic patterns." Below this is a "Login" form. The form has two input fields: "Email" with the value "arun@gmail.com" and "Password" with masked characters "*****". Below the password field is a checkbox labeled "I agree to the Terms & Conditions and Privacy Policy". At the bottom of the form is a blue "LOGIN" button. A link "Don't have an account? Register here" is visible at the bottom right of the form area.

Token Access

A successful login returns a bearer token that allows access to protected dashboard modules.

Authentication Form

The login page collects registered user credentials and submits them to the backend authentication service.

Validation Feedback

Users receive clear responses when credentials are missing, invalid, or rejected by the API.

Session Control

The stored token supports secure continuity across document, shipment, and analytics workflows.

Login Workflow

Credential verification, token creation, and client-side persistence are all part of the login process.

Protected Entry

This page acts as the controlled gateway into the operational dashboard and its secured routes.

Register Page

Professional analysis for plagiarism and AI-generated logic patterns.

Register

Email

Password

☒ I agree to the [Terms & Conditions](#) and [Privacy Policy](#)

REGISTER

Input Fields

The form captures the required identity information needed for protected project access.

Account Creation

The register page creates a new user account with username, email, and password fields.

Password Security

Passwords are sent to the backend and stored only after secure hashing during registration.

Duplicate Checks

The backend prevents duplicate usernames and duplicate email addresses from being registered.

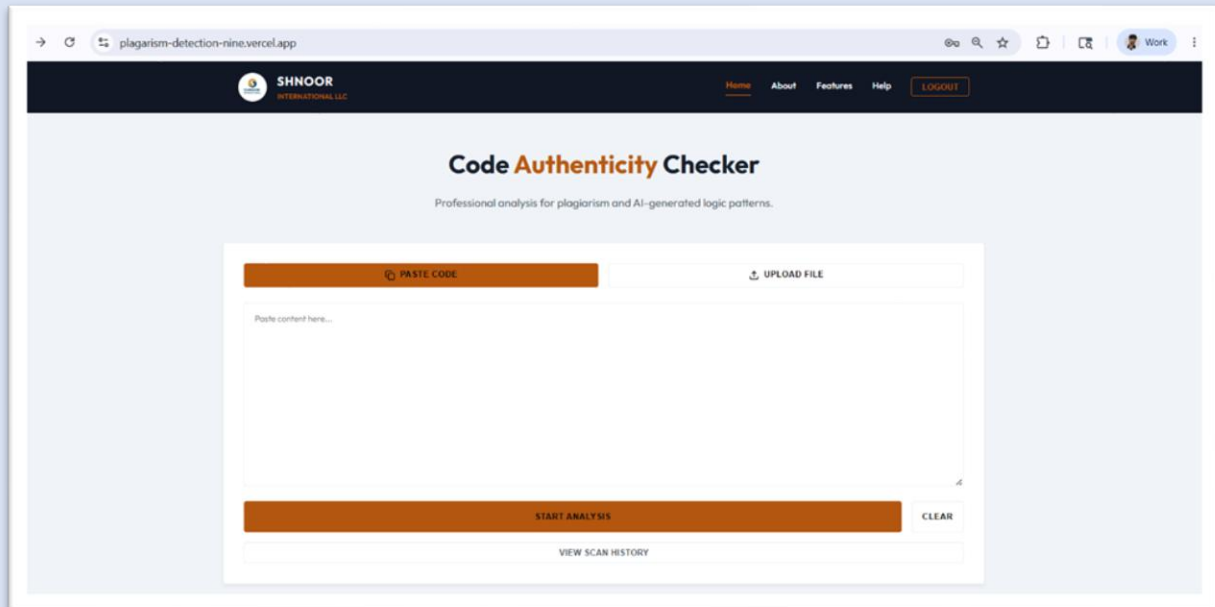
Onboarding Flow

This page supports clean onboarding into the AI Import Export system without manual admin setup.

User Setup

Successful registration prepares the user for immediate use of dashboard-based trade workflows.

Home Dashboard



Workspace Overview

The primary Home tab combines the text/file scanner, real-time result presentation, and optional scanning history access within a single, unified operational surface.

Navigation Modules

Users can seamlessly navigate between home, about, features, help, login, register, terms, and privacy views through dynamic React state switching in the App Shell.

Operational Focus

The dashboard strictly centers user attention on "Code Authenticity" analysis, allowing instant transitions between plagiarism checks and AI-generation report reviews.

App Shell Architecture

The persistent Header, dynamic hero copy, and lazy-loaded content switching logic create an intuitive, high-performance structure for all user journeys.

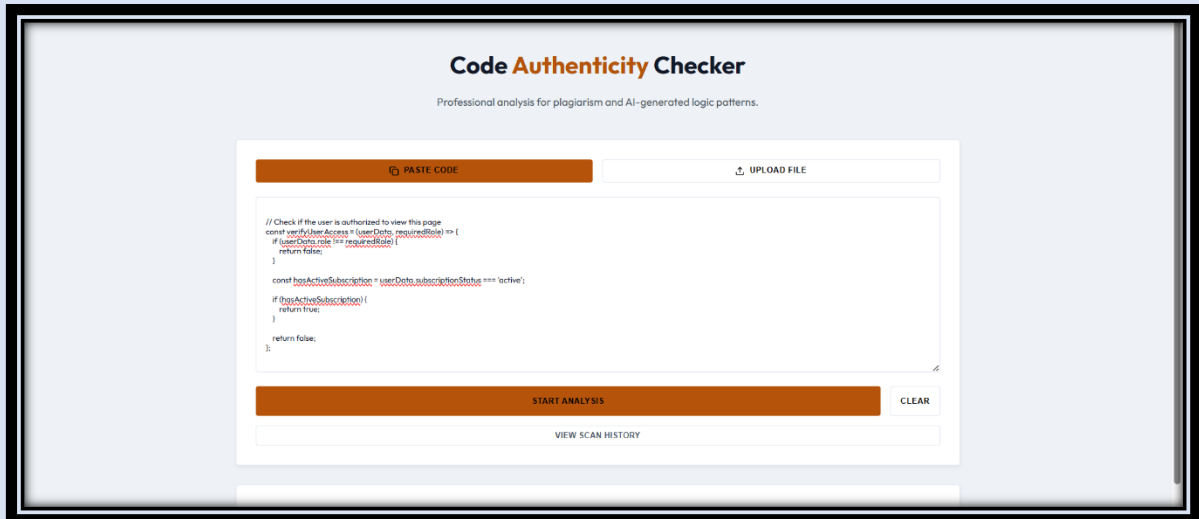
Dashboard Workflow

This central view functions as the primary launch point for executing new text/file scans, viewing detailed modal reports, and retrieving historical data.

Responsive Access

Desktop and mobile layouts dynamically adapt through shared CSS container helpers, ensuring the scanning workspace remains completely readable and functional across all device sizes.

Scanner Module



Paste Code

Users can paste raw source text directly into the dedicated analysis area, submitting it instantly to the Node.js backend for originality processing.

Upload File

The upload interface accepts user-selected documents, displays the active file name, and securely sends the data via a multipart request for server-side extraction and scanning.

Dual Input Flow

The React interface cleanly toggles between direct text review and file-based analysis, maintaining a unified, single-click scanning workflow regardless of the input method.

Focused UX : The module keeps the interaction streamlined, minimizing visual clutter so users can transition effortlessly from providing input to viewing the final report modal.

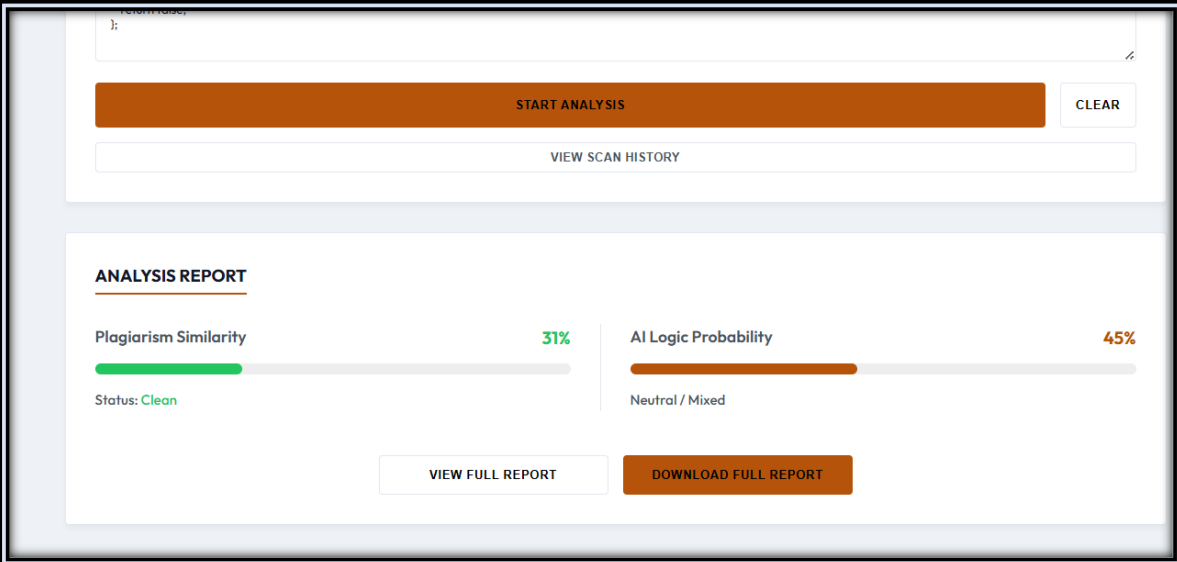
Action Controls

"START ANALYSIS," "CLEAR," and "VIEW SCAN HISTORY" buttons are strategically grouped, providing users with a compact, efficient, and intuitive operational surface.

Loading Feedback

A dynamic spinning loader state temporarily replaces the analysis button during active processing, clearly communicating system status while the AI engine evaluates the content.

Plagiarism Analysis



The screenshot shows a web interface for plagiarism analysis. At the top, there is a text input field with a small icon on the right. Below the input field are two buttons: a large orange "START ANALYSIS" button and a smaller white "CLEAR" button. Underneath these is a white button labeled "VIEW SCAN HISTORY". The main section is titled "ANALYSIS REPORT". It displays two metrics: "Plagiarism Similarity" at 31% with a green progress bar and a "Status: Clean" label, and "AI Logic Probability" at 45% with an orange progress bar and a "Neutral / Mixed" label. At the bottom of the report section are two buttons: "VIEW FULL REPORT" and "DOWNLOAD FULL REPORT".

Similarity Engine

The Node.js plagiarism service compares the newly submitted text against previous document scans using advanced string-similarity based mathematical scoring.

Personal Review Context

Authenticated scans strictly align with the user's isolated sandbox, ensuring new documents are only compared against their own private, historical scanning records.

Match Visibility

When similarity exceeds the baseline threshold, the result view presents a specific matched preview snippet, giving users immediate context around the detected overlap.

Status Outcome

Raw percentage scores are automatically converted into explicit, readable statuses (e.g., "Potential Plagiarism Detected" or "Clean") to help reviewers interpret authenticity quickly.

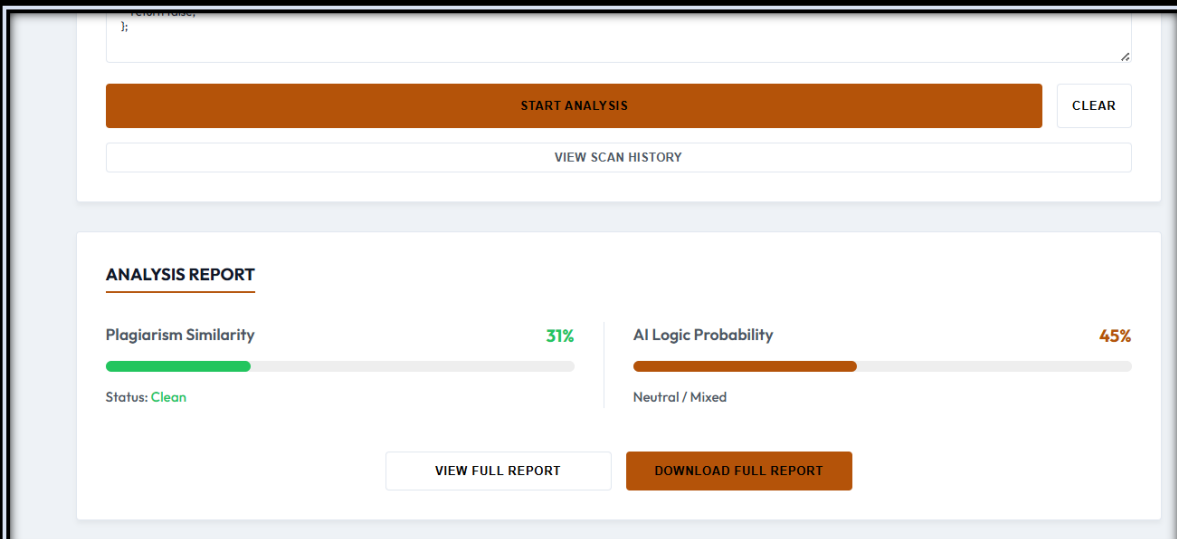
Content Preview Logic

Input, normalization, feature transformation, and model prediction are exposed through this page.

Review Confidence

By combining numerical scores with contextual snippets, this module transforms raw algorithm logic into a definitive operational outcome for end-user decision-making.

AI Logic Detection



Heuristic Intelligence

The custom AI detection service evaluates source code structure, commenting styles, vocabulary patterns, and deep statistical signals using regex-based pattern matching.

Pattern Dataset

Configurable rules stored securely inside `ai_patterns.json` support maintainable, weighted scoring behavior, allowing you to update the dataset logic without altering the core service interface.

Text Statistics

Mathematical evaluations of line-length variance (standard deviation) and vocabulary richness dynamically contribute to the final probability score and confidence rating.

Readable Labels

The platform automatically translates the final numerical AI probability into clear, understandable labels (e.g., "Highly Likely AI Generated" or "Likely Human Written").

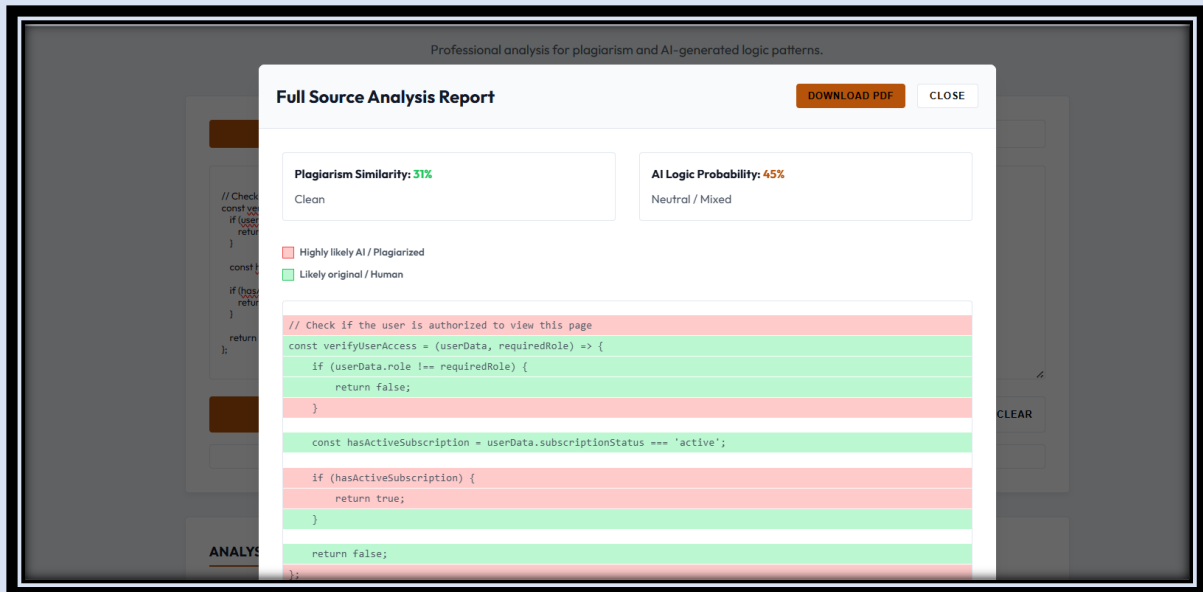
Integrated Workflow

The Node.js backend seamlessly aggregates the AI detection results and the plagiarism analysis into a single, unified JSON report payload for the frontend to render..

Decision Support

By evaluating both structural heuristics and statistical variances, this module helps reviewers confidently assess logic-writing styles with consistent, product-facing metrics.

Analysis Report



Score Presentation

Interactive Result Cards display the exact plagiarism percentage and AI logic probability through clear numeric metrics and dynamic, color-coded text indicators.

Progress Visualization

Animated CSS progress bars strengthen readability by visually communicating the relative magnitude of each reported score before the user dives into the full text analysis.

Full Report Modal

Users can launch the full ReportModal overlay for a deeper review of the source content, featuring deterministic, heuristic line-level highlighting (red for suspect lines, green for original).

Report Actions

The modal interface supports a native, CSS-optimized print-driven export path (DOWNLOAD PDF), allowing users to effortlessly generate PDF reports for sharing or archiving without complex external libraries.

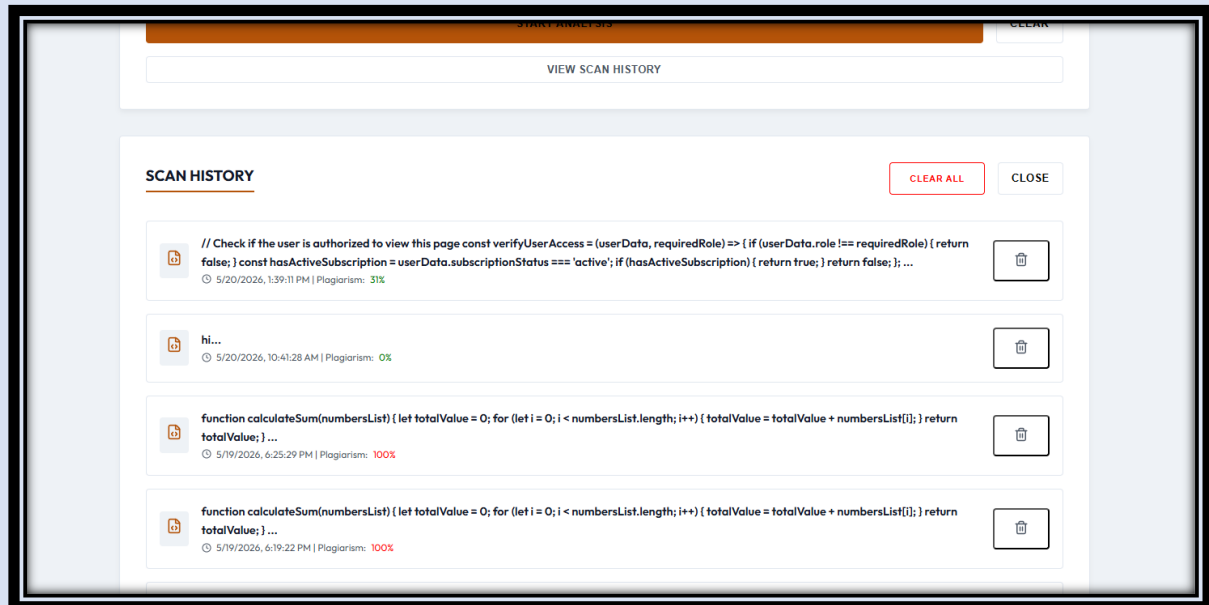
Status Messaging

Readable textual labels and contextual legend keys translate raw backend scoring data into an easy-to-understand, color-coded presentation layer for immediate visual comprehension.

Professional Review

By merging strict mathematical scoring with visual code-line rendering, this module gives the application a highly professional, reviewer-facing presentation experience for all scan outcomes.

Scan History



Stored Reviews

The interactive history view presents previously analyzed scans alongside formatted timestamps, exact text previews, and dynamically color-coded plagiarism scores.

Private Records

Database history access is strictly tied to the authenticated user's JWT identity via Node.js, ensuring completely private and isolated review experiences.

Single Delete

Users can permanently remove specific scan records from the PostgreSQL database using the intuitive Trash icon, allowing them to precisely curate their stored review list.

Clear All

A dedicated "CLEAR ALL" utility function supports a complete, one-click database history reset for accounts that require a completely fresh review cycle.

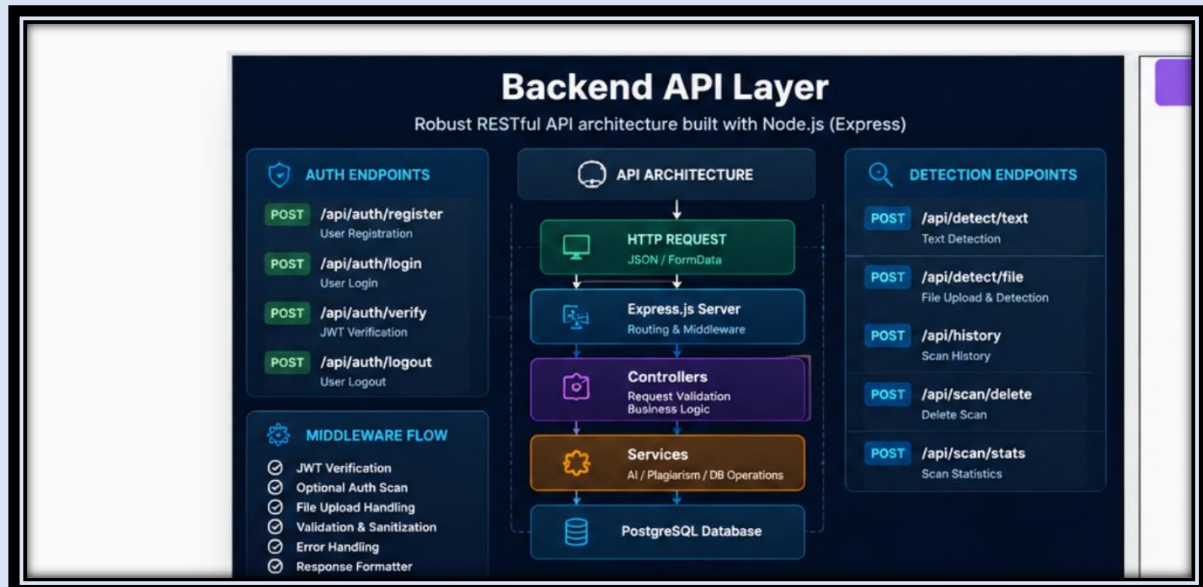
Timeline Utility

This centralized timeline enables users to instantly retrieve past authenticity data without incurring the processing time of resubmitting the same source content.

Operational Continuity

By preserving prior authenticity reviews in a highly accessible React component format, this module significantly reduces friction for repeat analysis and historical audits.

Backend API Layer



Auth Endpoints

The Express.js backend exposes secure registration and login endpoints, utilizing secure password hashing and JSON Web Tokens (JWT) for robust session workflows.

Detection Endpoints

Dedicated REST APIs support direct text parsing, multipart file upload analysis, user-scoped history retrieval, single record deletion, and complete database history resets.

Middleware Flow

Custom JWT verification middleware strictly protects private history routes, while the authOptional middleware enables flexible, unauthenticated access for public scanning.

Controller Layer

Modular Node.js controllers coordinate request validation, execute the AI/Plagiarism services, perform PostgreSQL database queries, and return structured JSON payloads.

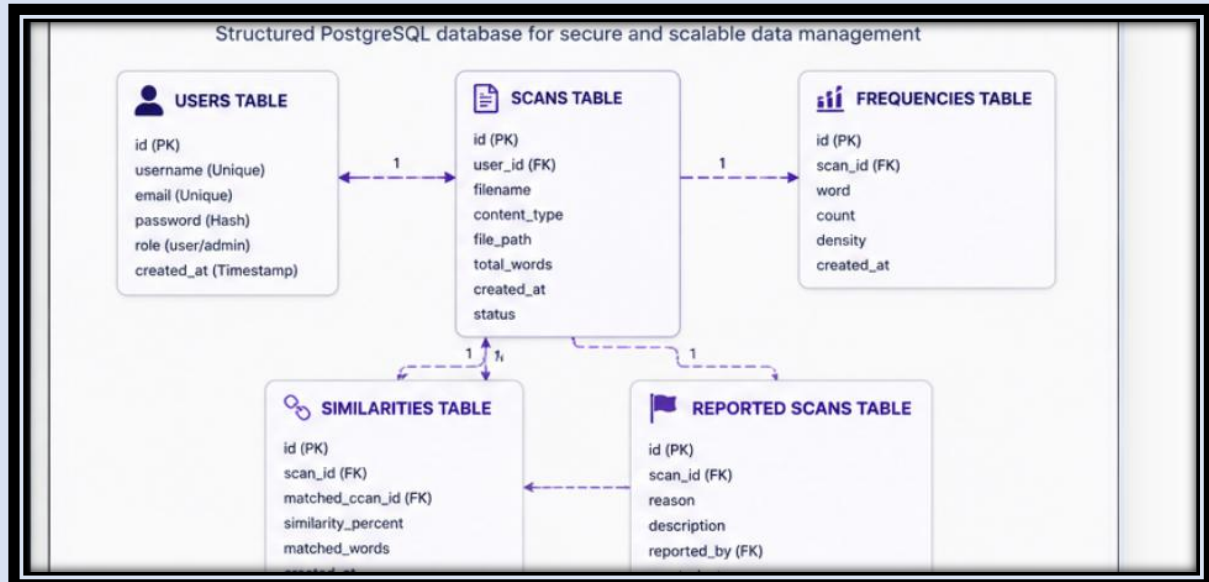
Business Routing

The Express routing architecture keeps authentication logic strictly separated from core scanning logic (detectionController.js), ensuring highly maintainable long-term growth.

Integration Path

Frontend Axios-based utility functions seamlessly connect the React user interface to the Node.js backend through a predictable, asynchronous API request structure.

Database Design



Users Table

The PostgreSQL users table acts as the authentication ledger, securely storing the unique account identity (email), cryptographically hashed passwords, and created_at timestamps required for JWT access.

Scans Table

The core scans table securely stores the content_type, optimized text previews, mathematical plagiarism_scores, ai_score probabilities, and original file_path metadata for every processed document.

Relational Link

A strict Foreign Key relationship (user_id REFERENCES users(id) ON DELETE CASCADE) tightly couples each scan to a specific user, enabling isolated, account-specific history and clean data persistence.

Timestamp Support

Default NOW() created_at values natively support fast, recency-based SQL retrieval (e.g., ORDER BY created_at DESC) and power the timeline-style frontend history rendering.

Content Traceability

Storing optimized string previews directly in the database helps power rapid repeat reviews, fast string-similarity comparisons, and immediate UI history readability without needing to load full files.

Operational Persistence

This relational SQL database design robustly supports both highly-secure user access management and strict, long-term analysis-result continuity.

Authentication & Security



JWT Workflow

State-of-the-art JSON Web Token (JWT) verification secures protected routes, strictly aligning user-bound features, like the isolated scanning sandbox, with active authenticated sessions.

Password Protection

Heavyweight bcrypt cryptographic hashing strengthens account protection, safely salting and storing credentials in PostgreSQL during registration and sign-in verification.

Protected Routes

Sensitive user profile and history-related API endpoints are rigorously gated through mandatory JWT validation middleware to prevent unauthorized data access.

Optional Auth Scan

The custom authOptional.js middleware supports flexible participation, allowing public anonymous scans while seamlessly attaching database records to a user_id when a valid token exists.

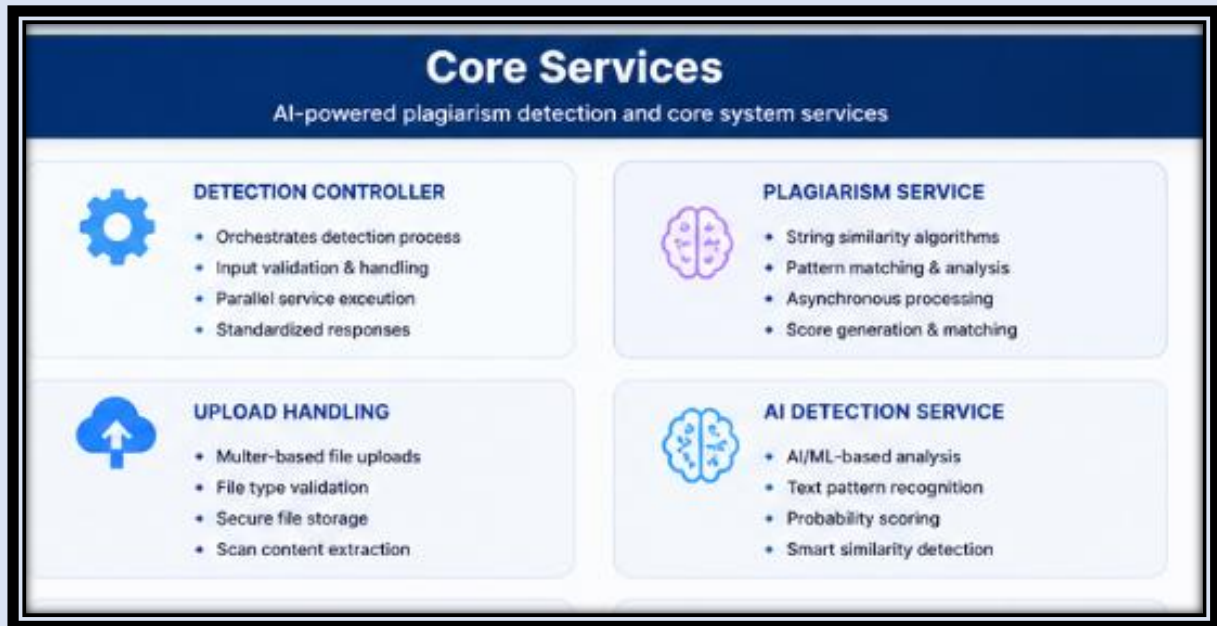
Authorization Header

The React frontend automatically and securely attaches the Bearer token to the Authorization HTTP header for all relevant API calls using a shared Axios interceptor.

Trusted Experience

These layered cryptographic mechanisms create a highly dependable, enterprise-grade foundation for secure text analysis workflows and private historical record access.

Core Services



Detection Controller

The detectionController acts as the primary orchestrator, managing input validation, parallel service execution, PostgreSQL record creation, and standardized JSON response formatting.

Plagiarism Service

The modular plagiarismService isolates the string-similarity logic into a highly reusable, asynchronous unit strictly focused on originality score generation and historical match context.

Upload Handling

A secure multer-backed middleware intake actively supports multipart file-based workflows, ensuring that file uploads seamlessly align with the exact same reporting pipeline as pasted content.

Structured Design :By strictly separating controllers from services and database configuration, these core modules give the Node.js project a highly maintainable, scalable, and professional architectural shape

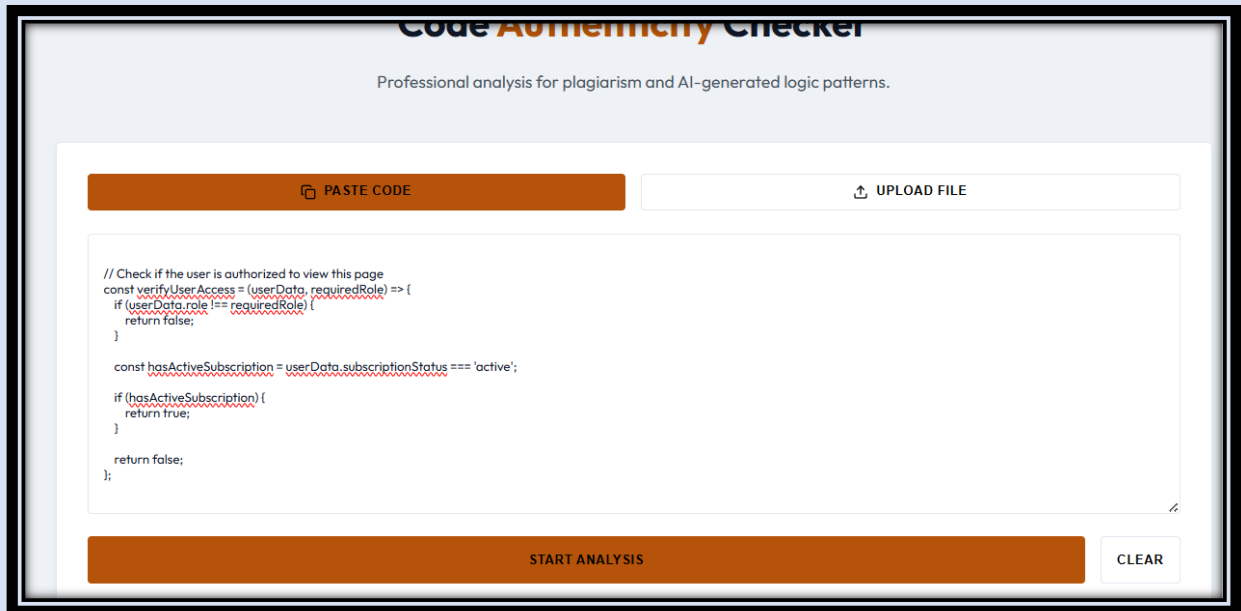
AI Detection Service

The aiDetectionService encapsulates deeply configurable, regex-driven AI heuristics, mathematically converting raw text variances into definitive, user-facing probability outcomes.

Database Config

The config/db.js utility centralizes pooled PostgreSQL connections via the pg library, ensuring efficient, highly concurrent database access across all route handlers and services.

User Experience



Manual Review Flow

The React application architecture supports rigorous end-to-end manual checks across the entire JWT sign-in process, document scanning, modal report viewing, and database history interactions.

Responsive Layout

The UI dynamically adjusts grid structures, flexbox button groupings, and text layouts for mobile screens through shared, root-level CSS container rules.

Usability Focus

User actions are direct, button labels are explicit (e.g., "START ANALYSIS"), and the entire workflow stays strictly centered on resolving text authenticity outcomes.

Consistent Theme

The application styling leverages a strict CSS variable system (var(--primary), var(--text-main)) to ensure a highly consistent, professional corporate look across all views and components.

Clear Presentation

The platform communicates complex backend data seamlessly through distinct percentage values, color-coded status labels, animated progress bars, and deterministic report legends.

Product Experience

Combined, these UX qualities ensure the system feels incredibly usable, remarkably modern, and intentionally purpose-built for high-stakes plagiarism and AI analysis.

Conclusion

Project Summary

The React application architecture supports rigorous end-to-end manual checks across the entire JWT sign-in process, document scanning, modal report viewing, and database history interactions.

Technical Foundation

The UI dynamically adjusts grid structures, flexbox button groupings, and text layouts for mobile screens through shared, root-level CSS container rules.

Product Value

User actions are direct, button labels are explicit (e.g., "START ANALYSIS"), and the entire workflow stays strictly centered on resolving text authenticity outcomes.

Growth Potential

The application styling leverages a strict CSS variable system (var(--primary), var(--text-main)) to ensure a highly consistent, professional corporate look across all views and components..